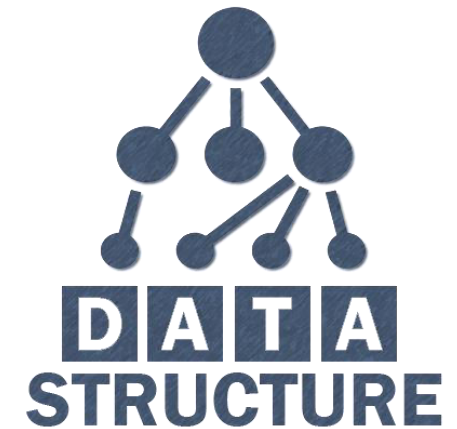


Unit-1

Introduction to Data Structure



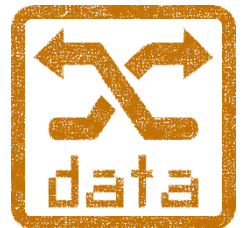


Outline

- Data Management concepts
- Data types
 - Primitive
 - Non-primitive
- Types of Data Structures
 - Linear Data Structures
 - Non Linear Data Structures
- Performance Analysis and Measurement
- Time analysis of algorithms
- Space analysis of algorithms

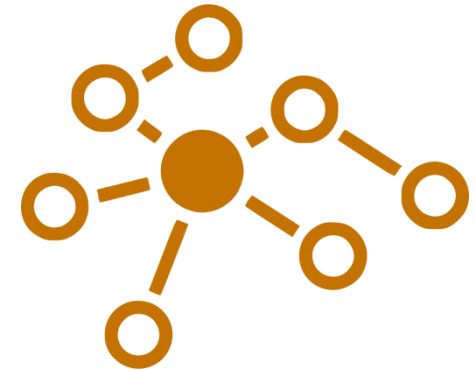
What is Data?

- ▶ **Data** is the basic fact or entity that is utilized in calculation or manipulation.
- ▶ There are two different **types of data** **Numeric** data and **Alphanumeric** data.
- ▶ When a programmer collects such type of data for **processing**, he would require **to store them in computer's main memory**.
- ▶ The process of storing data items in computer's main memory is called **representation**.
- ▶ **Data** to be processed must be **organized in a particular fashion**, these organization leads to structuring of data, and hence the mission to study the Data Structures.



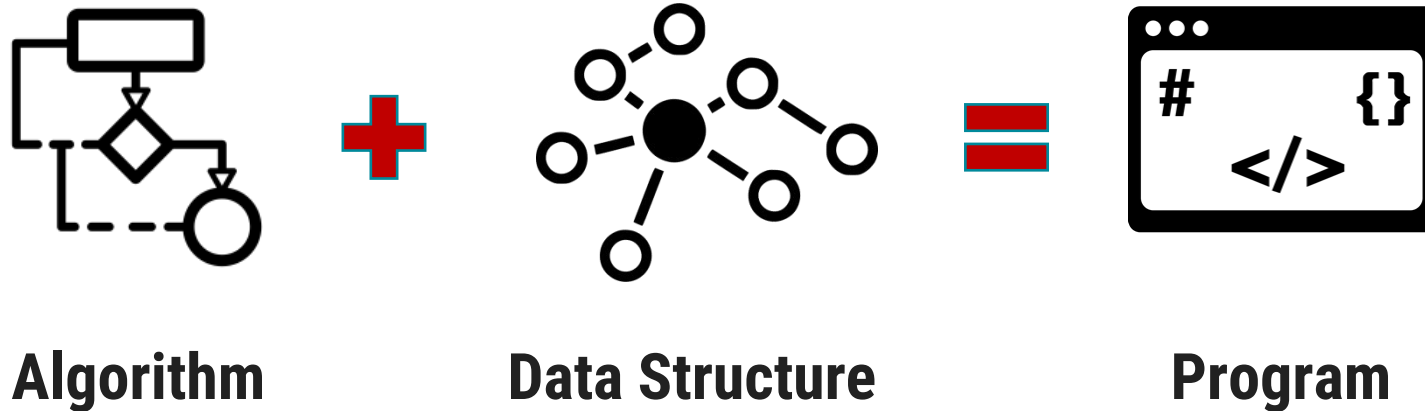
What is Data Structure?

- ▶ **Data Structure** is a representation of the logical relationship existing between individual elements of data.
- ▶ In other words, a data structure is a **way of organizing all data items** that **considers** not only the **elements stored** but also their **relationship to each other**.
- ▶ We can also define data structure as a **mathematical or logical model** of a particular **organization of data items**.
- ▶ Data Structure mainly specifies the following four things
 - Organization of Data
 - Accessing Methods
 - Degree of Associativity
 - Processing alternatives for information

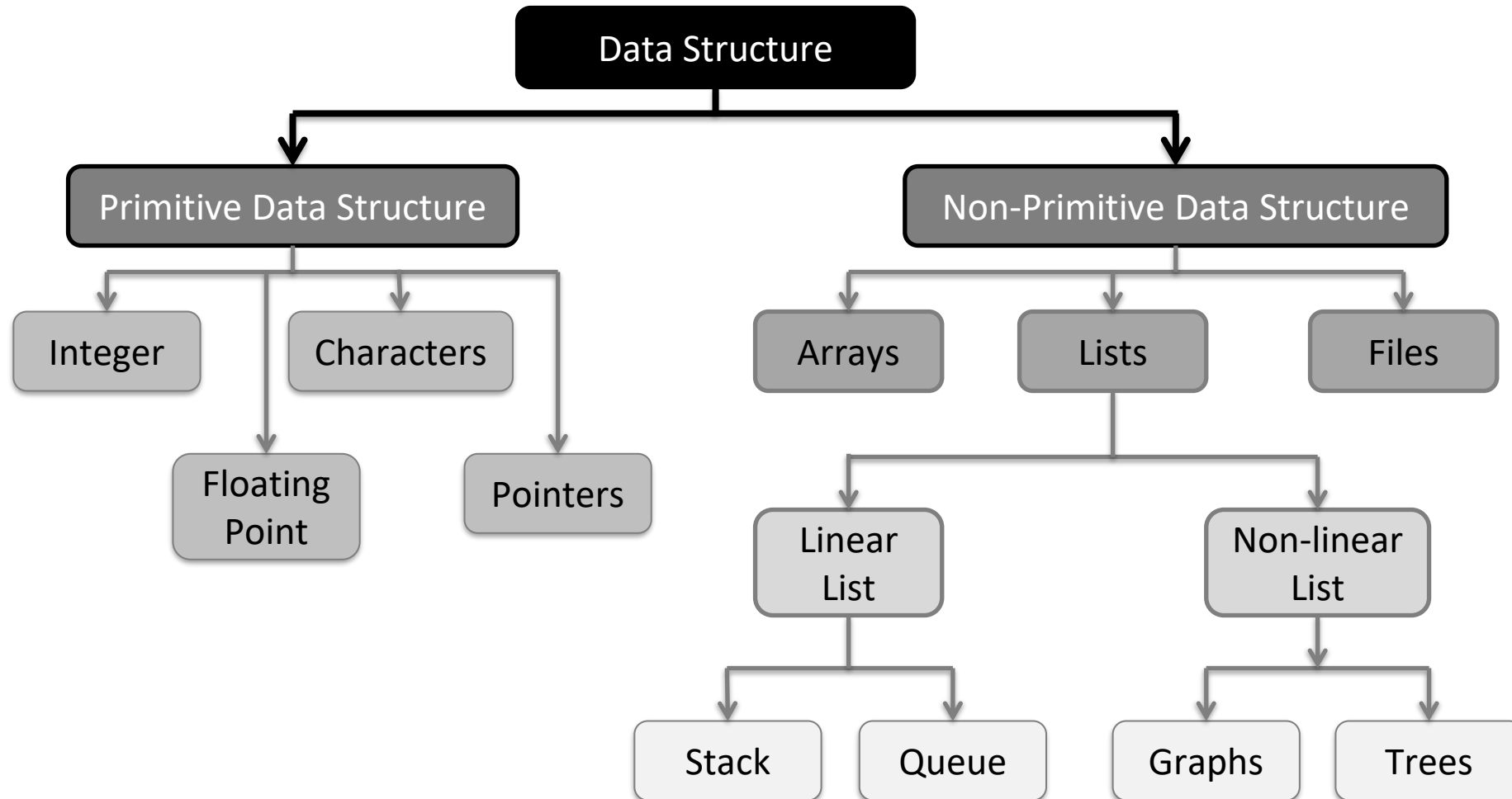


What is Data Structure? Cont..

- ▶ The **representation** of a particular data structure in the memory of a computer is called **Storage Structure**.
- ▶ The storage structure **representation** in auxiliary memory is called as **File Structure**.



Classification of Data Structure



Primitive / Non-primitive data structures

▶ Primitive data structures

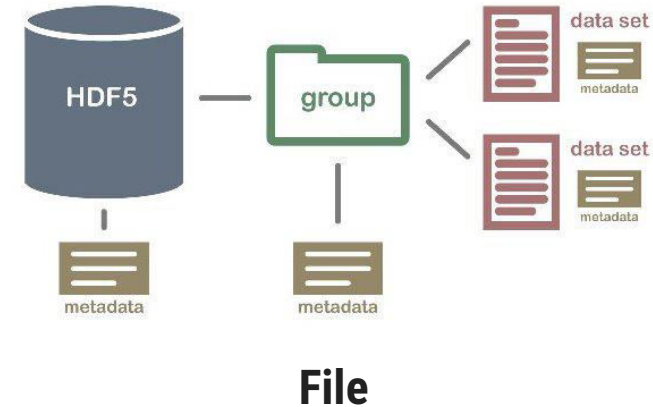
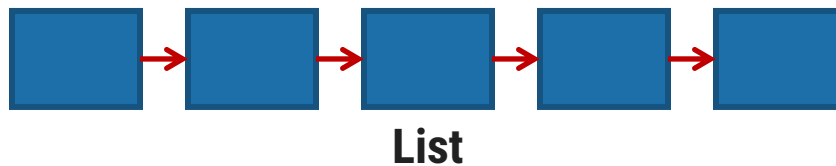
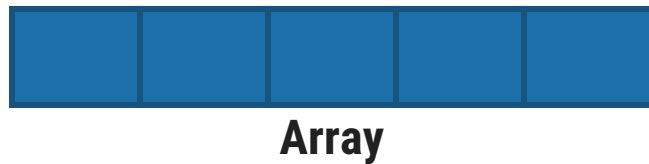
- Primitive data structures are basic structures and are directly operated upon by machine instructions.
- *Integers, floats, character* and *pointers* are examples of primitive data structures.

▶ Non primitive data structure

- These are derived from primitive data structures.
- The non-primitive data structures emphasize on structuring of a group of homogeneous or heterogeneous data items.
- Examples of Non-primitive data type are *Array, List*, and *File*.

Non primitive Data Structure

- ▶ **Array:** An array is a fixed-size sequenced collection of elements of the same data type.
- ▶ **List:** An ordered set containing variable number of elements is called as Lists.
- ▶ **File:** A file is a collection of logically related information. It can be viewed as a large list of records consisting of various fields.



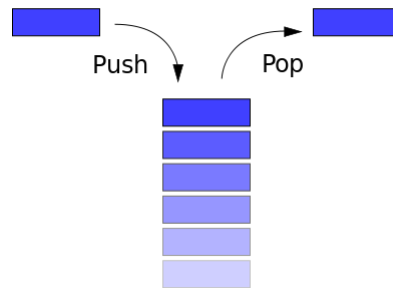
Linear / Non-Linear data structure

► Linear data structures

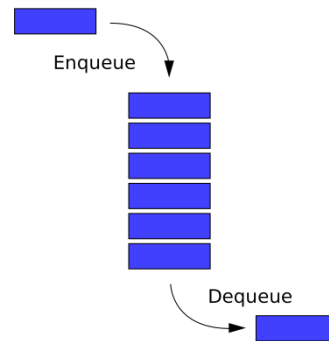
- ➔ A data structure is said to be Linear, if its elements are connected in linear fashion by means of logically or in sequence memory locations.
- ➔ Examples of Linear Data Structure are **Stack** and **Queue**.

► Nonlinear data structures

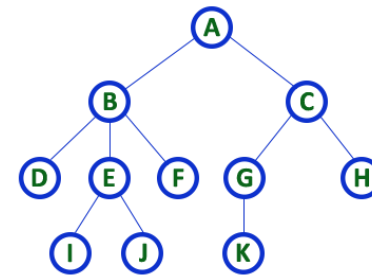
- ➔ Nonlinear data structures are those data structure in which data items are not arranged in a sequence.
- ➔ Examples of Non-linear Data Structure are **Tree** and **Graph**.



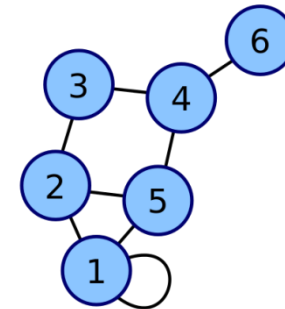
Stack



Queue



Tree



Graph

Operations of Data Structure

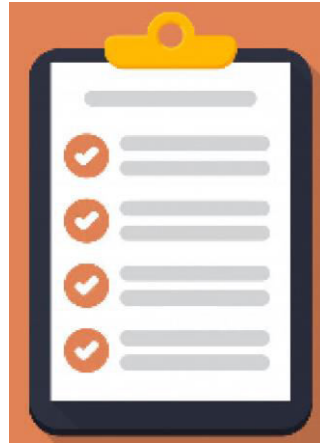
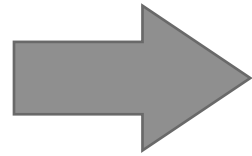
- ▶ **Create:** It results in reserving memory for program elements.
- ▶ **Destroy:** It destroys memory space allocated for specified data structure.
- ▶ **Selection:** It deals with accessing a particular data within a data structure.
- ▶ **Updation:** It updates or modifies the data in the data structure.
- ▶ **Searching:** It finds the presence of desired data item in the list of data items.
- ▶ **Sorting:** It is a process of arranging all data items in a data structure in a particular order.
- ▶ **Merging:** It is a process of combining the data items of two different sorted list into a single sorted list.
- ▶ **Splitting:** It is a process of partitioning single list to multiple list.
- ▶ **Traversal:** It is a process of visiting each and every node of a list in systematic manner.

What is an Algorithm?

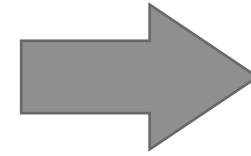
- A step-by-step procedure, to solve the different kinds of problems.
- Suppose, we want to make a Chocolate Cake.



Input
Ingredients



Process
Recipe



Output
Cake

- An unambiguous sequence of computational steps that transform the input into the output.

What is an Algorithm?

- A process or a set of rules to be followed to achieve desired output, especially by a computer.



- An algorithm is **any well-defined computational procedure** that takes some value, or a set of values as input and produces some value, or a set of values as output.

Characteristics of An Algorithm

- **Finiteness:** An algorithm must always terminate after a **finite number of steps**.
- **Definiteness:** Each step of an algorithm must be **precisely defined**.
- **Input:** An algorithm has **zero or more** inputs.
- **Output:** An algorithm must have **at least one** desirable output.
- **Effectiveness:** All the operations to be performed in the algorithm **must be sufficiently basic** and essential.
- **Correctness:** An algorithm must produce **correct output** on **every possible input instance** of the problem. It is the most important property of an algorithm.

Simple Multiplication Methods

1. American approach

$$\begin{array}{r} 981 \\ 1234 \\ \hline 3924 \\ 2943 \\ 1962 \\ 981 \\ \hline 1210554 \end{array}$$

2. English approach

$$\begin{array}{r} 981 \\ 1234 \\ \hline 981 \\ 1962 \\ 2943 \\ 3924 \\ \hline 1210554 \end{array}$$

Time and space analysis of algorithms

- ▶ Sometimes, there are more than one way to solve a problem.
- ▶ We need to learn how to compare the performance of different algorithms and choose the best one to solve a particular problem.
- ▶ While analyzing an algorithm, we mostly consider time complexity and space complexity.
- ▶ **Time complexity** of an algorithm quantifies the amount of time taken by an algorithm to run as a function of the length of the input.
- ▶ **Space complexity** of an algorithm quantifies the amount of space or memory taken by an algorithm to run as a function of the length of the input.
- ▶ Time & space complexity depends on lots of things like hardware, operating system, processors, etc.
- ▶ However, we don't consider any of these factors while analyzing the algorithm. We will only consider the execution time of an algorithm.

Calculate Time Complexity of Algorithm

- ▶ **Time Complexity** is most commonly **estimated** by **counting** the **number of elementary functions performed** by the algorithm.
- ▶ Since the algorithm's performance may vary with different types of input data,
 - ➔ hence for an algorithm we usually use the **worst-case Time complexity** of an algorithm because that is the maximum time taken for any input size.

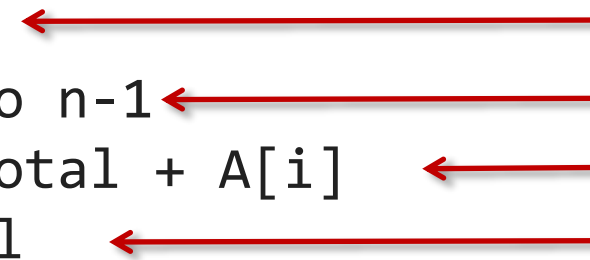
Calculating Time Complexity

► Calculate Time Complexity of Sum of elements of List (One dimensional Array)

SumOfList(A,n)  A is array, n is no of elements in array

```
{  
Line 1  total = 0  
Line 2  for i = 0 to n-1  
Line 3    total = total + A[i]  
Line 4  return total  
}
```

Line	Cost	No of Times
1	1	1
2	2	n + 1
3	2	n
4	1	1



$$T_{\text{SumOfList}} = 1 + 2(n+1) + 2n + 1$$

$$= 4n + 4$$
 We can neglect constant 4

$$= n$$

Time complexity of given algorithm is ***n*** unit time

Significance of Time Complexity

- As memory is available cheaply and in abundance, the space complexity of an algorithm is **not** considered as an **effective measure** for analysing an algorithm's performance.
- Running time of an algorithm with respect to the problem **instances of different sizes**, is often considered to be the **decisive measure** in differentiating between average algorithms and the optimal one.
- Thus, analysing the time complexity of an algorithm is **more important** than analysing the space complexity.

Efficiency of Algorithm

- ▶ The efficiency of an algorithm is a **measure of the amount of resources** consumed in solving a problem of size n .
- ▶ An algorithm must be **analyzed** to determine its resource usage.
- ▶ Two major computational resources are **execution time** and **memory space**.
- ▶ Memory Space requirement **can not be** compared directly, so the important resource is computational time required by an algorithm.
- ▶ To measure the **efficiency of an algorithm** requires to measure its execution time using any of the following approaches:
 1. **Empirical Approach:** To run it and measure how much processor time is needed.
 2. **Theoretical Approach:** Mathematically computing how much time is needed as a function of input size.

Time Complexity

- ▶ Time complexity of an algorithm quantifies the **amount of time taken by an algorithm** to run as a function of the length of the input.
- ▶ Running time of an algorithm depends upon,
 1. **Input Size**
 2. **Nature of Input**
- ▶ Generally time **grows** with the size of input, for example, sorting 100 numbers will take less time than sorting of 10,000 numbers.
- ▶ So, running time of an algorithm is usually measured **as a function of input size**.
- ▶ Instead of measuring actual time required in executing each statement in the code, we consider **how many times** each statement is executed.
- ▶ So, in theoretical computation of time complexity, running time is measured in terms of number of **steps/primitive operations** performed.

Linear Search

- ▶ Suppose, you are given a jar containing some business cards.
- ▶ You are asked to determine whether the name “Bill Gates” is in the jar.
- ▶ To do this, you decide to simply go through all the cards one by one.
- ▶ How long this takes?
- ▶ Can be determined by how many cards are in the jar, i.e., Size of Input.
- ▶ Linear search is a method for finding a particular value from the given list.
- ▶ The algorithm checks each element, one at a time and in sequence, until the desired element is found.
- ▶ Linear search is the simplest search algorithm.
- ▶ It is a special case of brute-force search.



Linear Search – Example

Search for **1** in given array

2	9	3	1	8
---	---	---	---	---

Comparing **value of i^{th} index** with the given element one by one, until we get the required element or end of the array

Step 1: $i=1$

2	9	3	1	8
---	---	---	---	---



$2 \neq 1$

Step 2: $i=2$

2	9	3	1	8
---	---	---	---	---



$9 \neq 1$

Step 3: $i=3$

2	9	3	1	8
---	---	---	---	---



$3 \neq 1$

Step 4: $i=4$

2	9	3	1	8
---	---	---	----------	---



Element found at i^{th} index, $i=4$

Linear Search - Algorithm

Input : Array A, element x

Output : First index of element x in A or -1 if not found

Algorithm: Linear_Search

for i = 1 to last index of A

if A[i] equals element x

return i

return -1

Linear Search - Analysis

► The required element in the given array can be found at,

1. E.g. 2: It is at the first position

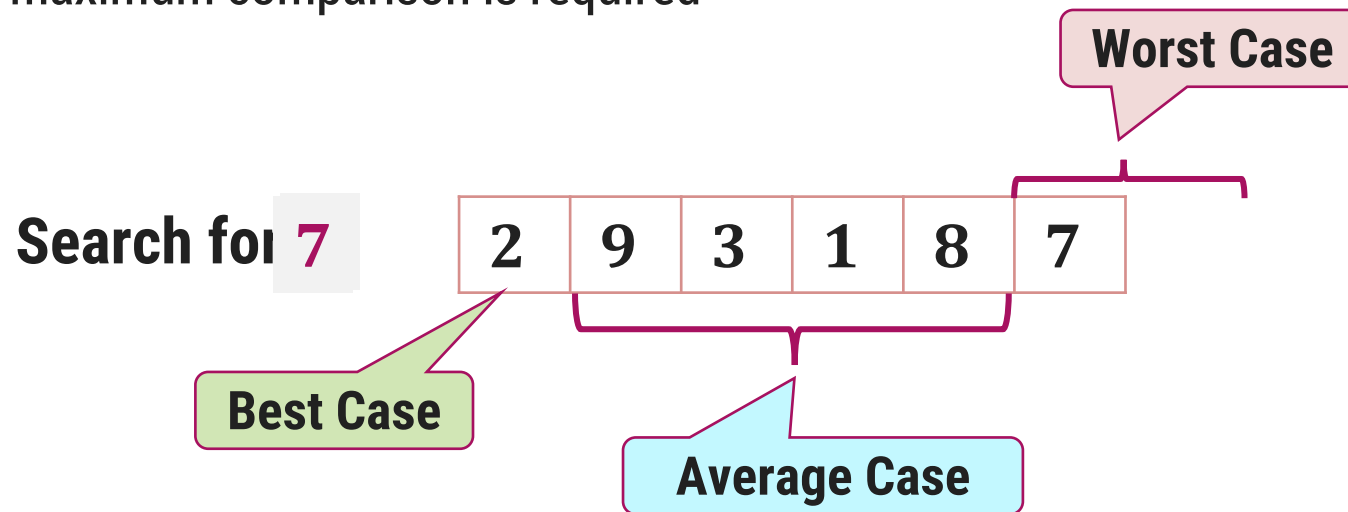
Best Case: minimum comparison is required

2. E.g. 3 or 1: Anywhere after the first position

Average Case: average number of comparison is required

3. E.g. 7: Last position or element does not found at all

Worst Case: maximum comparison is required



Linear Search - Analysis

- ▶ The required element in the given array can be found at,

Case 1: element 2 which is at the first position so minimum comparison is required

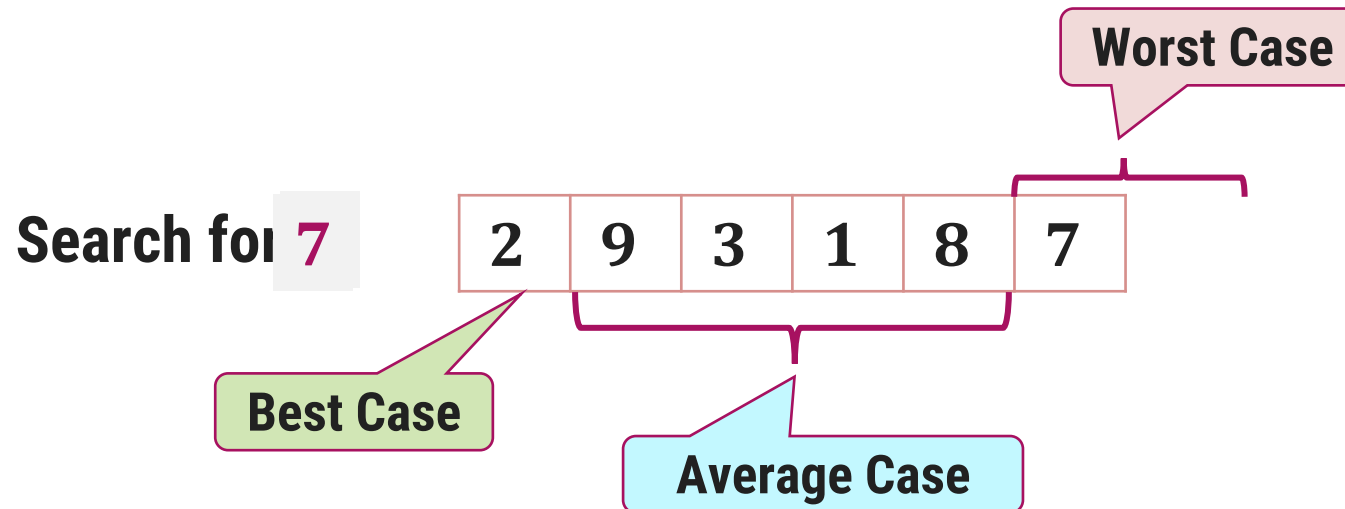
Best Case

Case 2: element 3 anywhere after the first position so, an average number of comparison is required

Average Case

Case 3: element 7 at last position or element does not found at all, maximum comparison is required

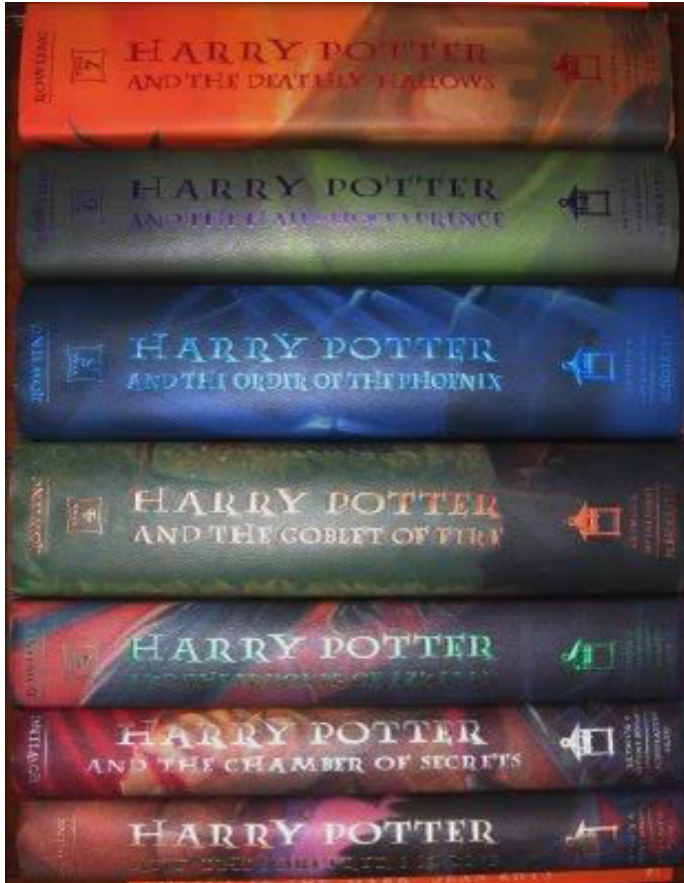
Worst Case



Analysis of Algorithm

Best Case	Average Case	Worst Case
Resource usage is minimum	Resource usage is average	Resource usage is maximum
Algorithm's behavior under optimal condition	Algorithm's behavior under random condition	Algorithm's behavior under the worst condition
Minimum number of steps or operations	Average number of steps or operations	Maximum number of steps or operations
Lower bound on running time	Average bound on running time	Upper bound on running time
Generally do not occur in real	Average and worst-case performances are the most used in algorithm analysis.	

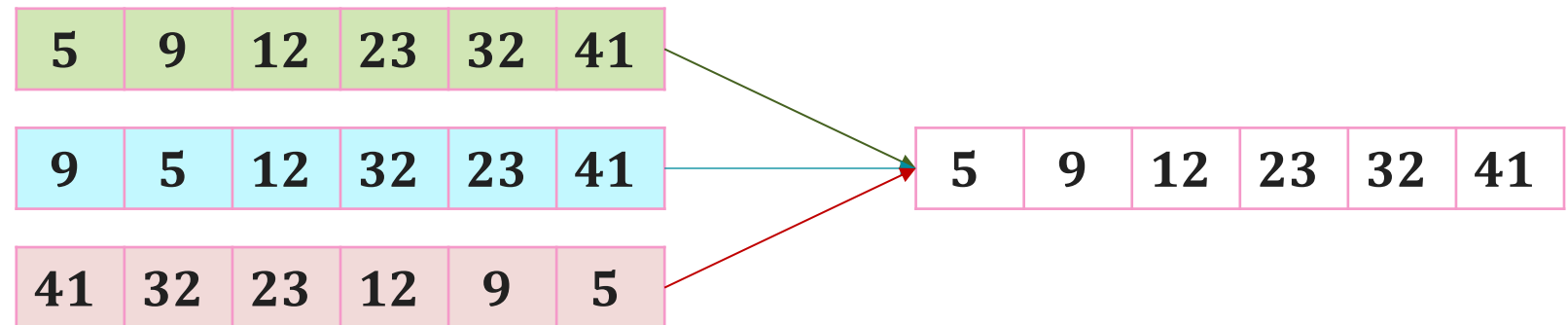
Book Finder Example



- Suppose, you are writing a program to find a book from the shelf.
- For any required book, it will start checking books one by one from the bottom.
- If you wanted Harry Potter 3, it would only take 3 actions (or tries) because it's the third book in the sequence.
- If Harry Potter 7 — it's the last book so it would have to check all 7 books.
- What if there are total 10 books? How about 10,00,000 books? It would take 1 million tries.

Number Sorting - Example

- ▶ Suppose you are sorting numbers in Ascending / Increasing order.
- ▶ The initial arrangement of given numbers can be in any of the following three orders.
 1. Numbers are already in required order, i.e., Ascending order
No change is required – **Best Case**
 2. Numbers are randomly arranged initially.
Some numbers will change their position – **Average Case**
 3. Numbers are initially arranged in Descending or Decreasing order.
All numbers will change their position – **Worst Case**



Number Sorting - Example

- ▶ Suppose you are sorting numbers in Ascending / Increasing order.
- ▶ The initial arrangement of given numbers can be in any of the following three orders.

Case 1: Numbers are already in required order, i.e., Ascending order
No change is required

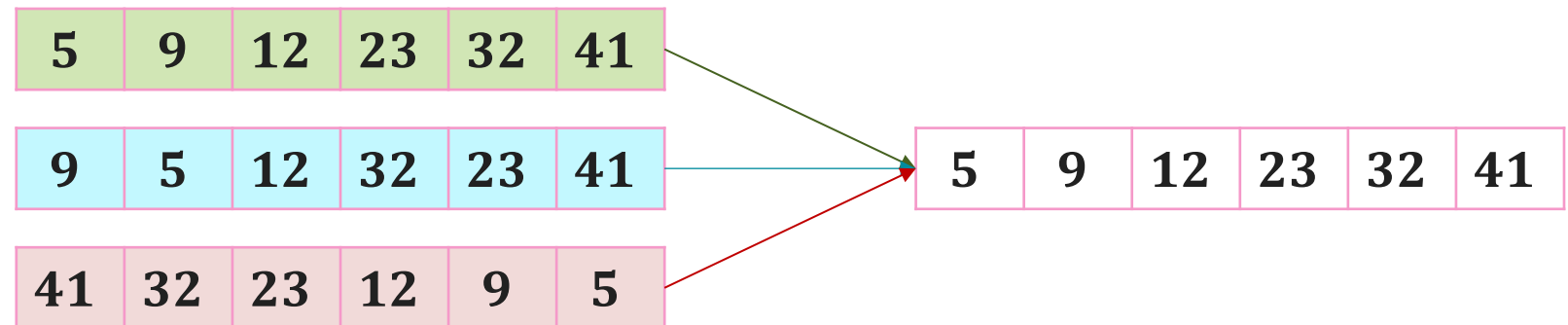
Best Case

Case 2: Numbers are randomly arranged initially. Some numbers will change their position

Average Case

Case 3: Numbers are initially arranged in Descending order so, all numbers will change their position

Worst Case



Best, Average, & Worst Case

Problem	Best Case	Average Case	Worst Case
Linear Search	Element at the first position	Element in any of the middle positions	Element at last position or not present
Book Finder	The first book	Any book in-between	The last book
Sorting	Already sorted	Randomly arranged	Sorted in reverse order

What is a Good Algorithm?

▶ Efficient

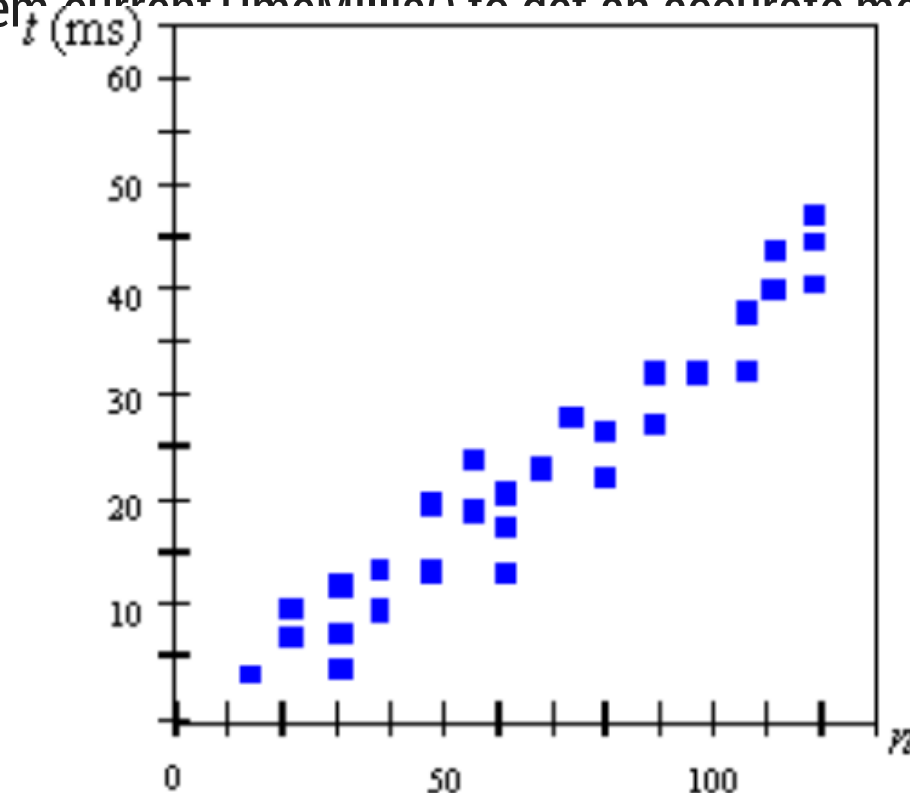
- ↳ Running time
- ↳ Space used

▶ Efficiency as a function of input size

- ↳ The number of bits in an input number
- ↳ Number of data elements(Numbers and Points)

Measuring the Running Time

- ▶ How should we measure the running time of an algorithm?
- ▶ Experimental Study
 - ➔ Write a program that implements the algorithm
 - ➔ Run the program with data sets of varying size and composition.
 - ➔ Use a method like `System.currentTimeMillis()` to get an accurate measure of the actual running time

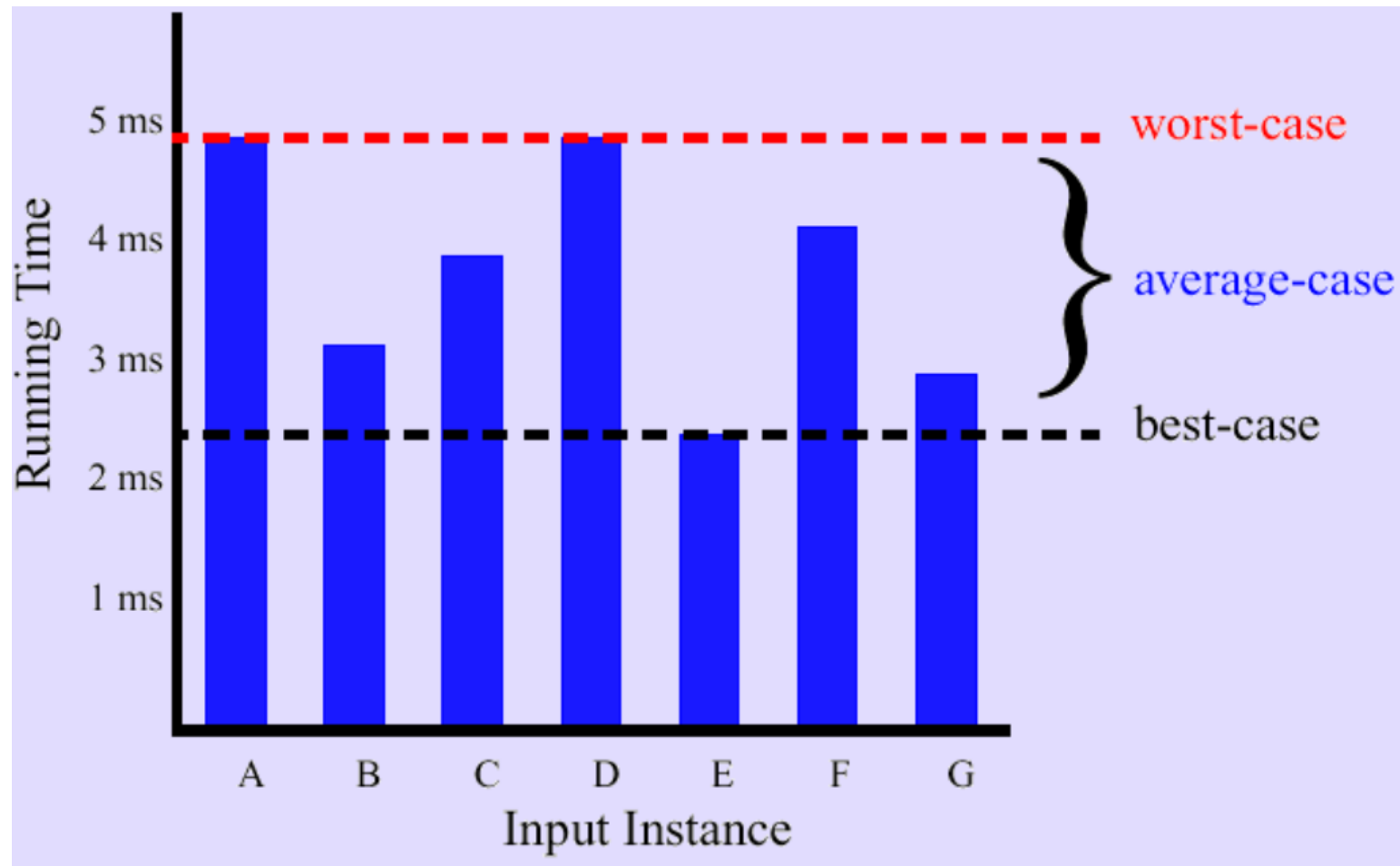


Limitations of Experimental Studies

- ▶ It is **necessary** to implement and **test** the algorithm in order to determine its running time.
- ▶ Experiments can be done only on a **limited set of inputs**, and may not be indicative of the running time on other inputs not included in the experiment.
- ▶ In order to **compare** two algorithms, the **same hardware and software environments** should be used.

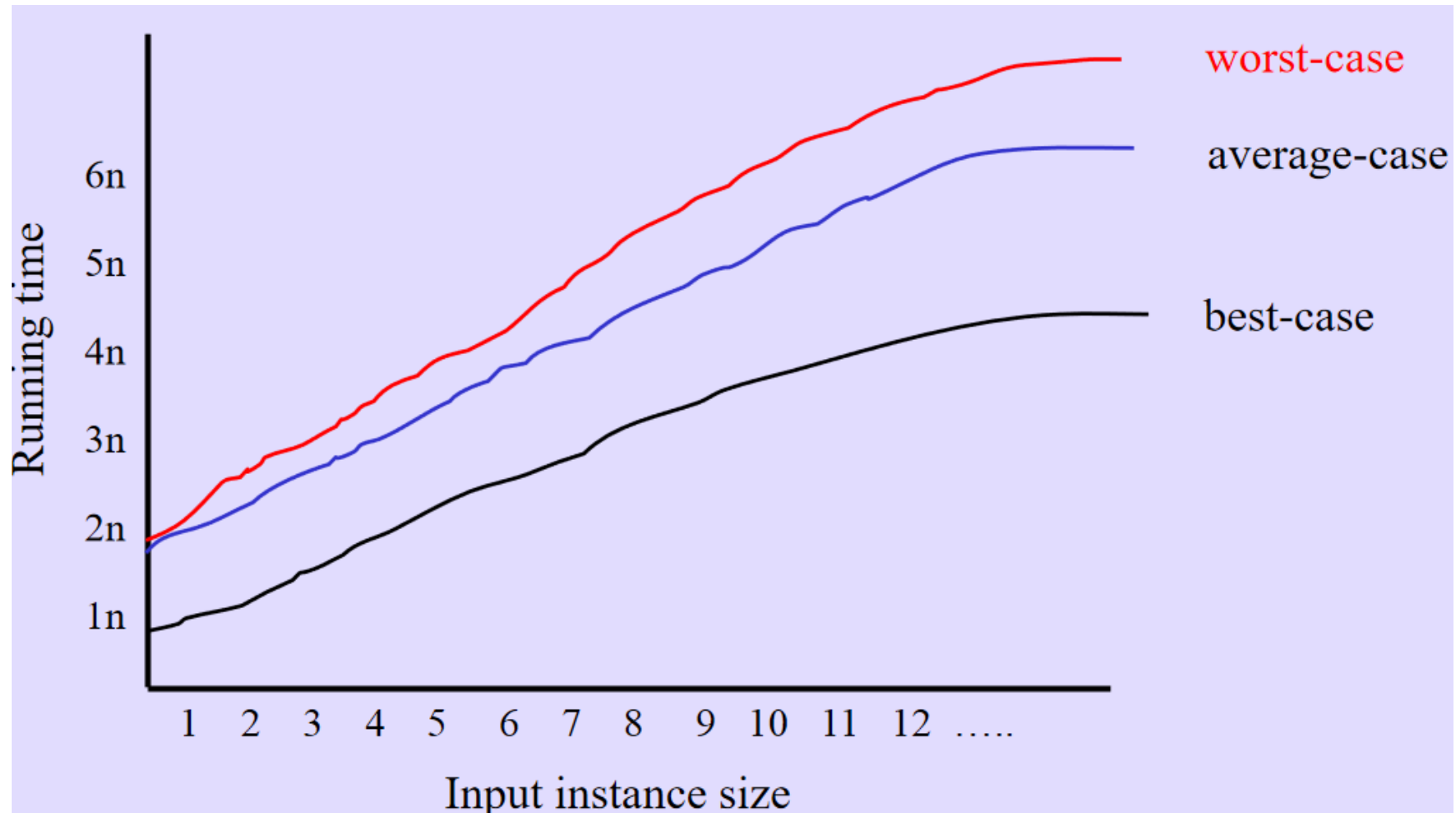
Best/Worst/Average Case

- For a specific size of input n , investigate running times for different input instances:



Best/Worst/Average Case

► For inputs of all sizes:



Googol-to-One Gear Ratio

- ▶ **Daniel Bruin** built a machine with 100 gears with a 10-to-1 gear ratio...meaning that the overall gear ratio is a googol-to-one. (A googol is 1 with 100 zeros.)

1 rotation of 1st gear = 10 second

10 rotation of i^{th} gear turns 1
rotation of $(i+1)^{\text{th}}$



Puzzles

- ▶ 1. You are provided with **10 identical** balls and a measuring instrument. 9 of the eight balls are equal in weight, and one of the 10 given balls is defective and weighs less. The task is to find the defective ball in minimum measurements.
- ▶ 2. Case: defective but don't know (weight less or more)
- ▶ 3. You are **blindfolded**, and **10 coins** are placed in front of you on the table. You are allowed to touch the coins but can't tell which way up they are by feel. You are told that there are 5 coins head up, and 5 coins tails up but not which ones are which.
- ▶ Can you make two piles of coins, each with the same number of heads up? You can flip the coins any number of times.

- ▶ 4. There is a room with a door (closed) and three light bulbs inside the room. Outside the room, there are three switches, connected to the bulbs. You may manipulate the switches as you wish, but once you open the door, you can't change them. All bulbs are in working condition, and you can open the door only once. Identify each switch with respect to its bulb.
- ▶ 5. There are **25 horses**, among which you need to find out the fastest 3 horses. You can conduct a race among at most 5 to find out their relative speed. At no point, can you find out the actual speed of the horse in a race. Find out the minimum no. of races that are required to get the top 3 horses.